

Recursion

CMPT 145

Copyright Notice

©2020 Michael C. Horsch

This document is provided as is for students currently registered in CMPT 145.

All rights reserved. This document shall not be posted to any website for any purpose without the express consent of the author.

Learning Objectives

After studying this chapter, a student should be able to:

- Derive recursive formulations for algorithms.
- Author, test, and debug recursive Python functions.
- Explain recursion in terms of the delegation metaphor.
- Describe how recursion is implemented in terms of functions calls and stack frames.
- Analyze simple recursive functions for time complexity.
- Analyze simple recursive functions for space complexity, including the use of the system stack.
- Determine whether a recursive or iterative solution is most appropriate for a problem. [Assessment, SDF-AD-6]

Dispelling concerns about recursion

- Recursion is a form of repetition based on function calls, instead of loops.
 - You may like loops better because you practice those more.
 - Recursion is easier than loops. Proof:
 - Recursion expresses a meaningful relationship.
 - Understanding relationships is what humans do best.
 - Therefore, recursion is what humans do best.
- Q.E.D.

Recursion

- Definitions that refer to themselves are said to be “recursive”.
- Example:

$$n! = \begin{cases} 1 & \text{if } n = 0 \text{ (base case)} \\ n \cdot (n - 1)! & \text{if } n > 0 \text{ (inductive step)} \end{cases}$$

- Suppose we re-formulate the ! operation as a function:

$$factorial(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot factorial(n - 1) & \text{if } n > 0 \end{cases}$$

- It's easy to re-formulate this as a recursive function.

Recursive Algorithms

- Recursive functions are those that call themselves.
- Recall that each time a recursive call is made, that call gets its own copy of local variables.

Recursive Factorial Algorithm

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

Thinking recursively: The delegation metaphor

- A function call is like delegation. You "hire" a "delegate" to do some work for you.
- You give the delegate 3 things:
 1. Some data.
 2. Some instructions.
 3. A place to work.
- You wait until your delegate returns to you with the answer you asked for.
- If you gave your delegate a copy of the same instructions you are using, it's a recursive function.

Note:

It does not matter at all that your delegate is using the same instructions you are using.

Designing Recursive Algorithms

Recursive Structure

Every recursive function is essentially a conditional.

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

Designing Recursive Algorithms

Base case(s)

At least one branch of the conditional has a very simple return.

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

Designing Recursive Algorithms

Recursion Property

The base case expresses a solution to the smallest problem that can be solved by the function.

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

Designing Recursive Algorithms

General (aka Recursive or Inductive) Case(s)

At least one branch of the conditional has a call to the function itself.

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

Designing Recursive Algorithms

Recursion Property

The recursive case expresses a true relationship between the problem you are given, and the solution to a smaller, related problem.

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

A template for recursive algorithms

```
1 def <recfun>(<data>)  
2     if <base case test>:  
3         <base case task>  
4     else:  
5         Calculate <new data> for smaller, related problem  
6         <subSolution> = Call <recfun>(<new data>)  
7         <solution> = Combine <subSolution> with <data>  
8     return <solution>
```

Sum of Squares

Recursive case

Calculate the sum of squares: $1^2 + 2^2 + 3^2 + \dots + n^2$, for $n > 0$

- Let

$$s(n) = 1^2 + 2^2 + 3^2 + \dots + n^2$$

- Rewrite using $n - 1$:

$$s(n - 1) = 1^2 + 2^2 + 3^2 + \dots + (n - 1)^2$$

...but only if $n > 1$ of course. What happens if $n = 1$?

- Subtract:

$$s(n) - s(n - 1) = n^2$$

because they have the same terms except the last one

- Rearrange:

$$s(n) = n^2 + s(n - 1)$$

Sum of Squares

Base case

Calculate the sum of squares: $1^2 + 2^2 + 3^2 + \dots + n^2$, for $n > 0$

- Let

$$s(n) = 1^2 + 2^2 + 3^2 + \dots + n^2$$

- Then the smallest n we can consider is $n = 1$
- So:

$$s(1) = 1^2 = 1$$

Sum of Squares

Use the template to build a function for Sum of Squares

Thinking recursively: Rules of thumb

Numerical Recursion:

- Parameters often include at least one integer.
- Base cases of $n = 0$ or $n = 1$ is very common.
- Smaller subproblem is often $n - 1$; sometimes you can use $n/2$.
- Building up a solution always uses *associativity*, e.g.,
 $1 + 2 + 3 = (1 + 2) + 3$ or $1 \times 2 \times 3 = (1 \times 2) \times 3$

Note:

The right choice is right because it fits your problem.

Example: Binary search

```
1 def bin_search(C, target, start, end):
2     """
3     Determine if target occurs in list C, between
4     indices start and end
5     Pre-conditions:
6     C: a list of data items in increasing order
7     target: the target key
8     start: first offset of C to be searched
9     end: last offset of C to be searched
10    Return: True if C contains the target
11    """
12    if end < start:
13        return False
14
15    mid = (start + end) // 2
16
17    if C[mid] == target:
18        return True
19    elif C[mid] < target:
20        return bin_search(C, target, mid+1, end)
21    else:
22        return bin_search(C, target, start, mid-1)
```

Thinking recursively: Understanding the problem

- Recursion always makes sense as a relationship.
- Relationships come from understanding the problem.
 - Draw diagrams.
 - Do short mathematical derivations.
 - Write down the solution to small examples.
 - Try to find relationships between problems of different sizes.
 - Staring at Python code is not helpful.

Exercise 1

N	$F(N)$
0	1
1	3
2	5
3	7
4	9
5	11

Complete the following equations:

(a) $F(5) = F(4) + \dots$

(b) $F(4) = F(3) + \dots$

(c) $F(N) = F(\dots) + \dots$

Exercise 2

N	$F(N)$
0	1
1	3
2	5
3	7
4	9
5	11

Generalize as a recursive function!

Exercise 3

N	$F(N)$
0	1
1	2
2	4
3	7
4	11
5	16

Complete the following equations:

(a) $F(5) = F(4) + \dots$

(b) $F(4) = F(3) + \dots$

(c) $F(N) = F(\dots) + \dots$

Generalize as a recursive function!

Exercise 4

N	$F(N)$
0	1
1	2
2	4
3	8
4	16
5	32

Complete the following equations:

(a) $F(5) = F(4) + \dots$

(b) $F(4) = F(3) + \dots$

(c) $F(N) = F(\dots) + \dots$

Generalize as a recursive function!

Exercise 5

Consider the following table:

S	F(S)
''	''
'd'	'd'
'cd'	'dc'
'bcd'	'dcb'
'abcd'	'dcba'

Complete the following equations:

(a) $F('abcd') = F('bcd') + \dots$

(b) $F('abc') = F('bc') + \dots$

(c) $F('ab') = F('b') + \dots$

Generalize as a recursive function!

Designing Recursive functions: Exercises

Write a recursive function for each of the following:

1. Calculate r^n for $n \geq 0$. Assume n is integer.
2. Calculates $1 - 2 + 3 - 4 + 5 \cdots n$
3. Determines if an integer n is even.
4. Determines if n is evenly divisible by m .

Our node ADT is recursive!

- This is not coincidence.
- Node sequences, like integers, are **inherently** recursive.
- A node sequence starting at `theNode` always contains a smaller sequence starting at `get_next(theNode)`!

Thinking recursively: Rules of thumb for Node chains

- Parameters include at least one node (not the List record)
- Base cases always include `theNode == None`
- Not uncommon to have multiple base cases, e.g. in search
- Smaller subproblem always follows the next field, e.g., `theNode.get_next()`. It's smaller because following the reference gets you closer to the end of the sequence.

Designing Recursive functions: Exercises

Write a recursive function for each of the following:

1. Displays the data values stored in a node-chain
2. Counts the number of nodes in a node-chain
3. Calculates the sum of the numbers stored in a node-chain

Assume that a node chain starts with the reference stored in the variable `chain`.

Thinking recursively

- Thinking recursively means understanding relationships in the problem.
- It would be a big mistake for anyone to try to understand recursion by simulating the computer.
- Still, we have to understand how recursion is implemented.
- Keep these two ideas separate. They are related, but not equivalent.
 1. Understanding recursive relationships.
 2. Understanding the implementation of recursion by a computer.

How does a function call work?

1. A **function call** **creates** a **"frame"**.
2. Parameters and local variables are names in the frame.
3. Each name has a place to store a reference to a value.
4. When the frame is created, it also stores a reference to the place where the return value should go.
5. When the **function returns**, the return value is saved, but the frame is discarded.

How does Recursion Work?

1. A **function call** **creates** a **"frame"**.
2. Parameters and local variables are names in the frame.
3. Each name has a place to store a reference to a value.
4. When the frame is created, it also stores a reference to the place where the return value should go.
5. When the **function returns**, the return value is saved, but the frame is discarded.

Recursion works because there is a chain of frames, forming a kind of stack. There is nothing special about a recursive function.

The System Stack, a.k.a the Call Stack

- In Python, frames created by a function call are stored in the heap.
- In some languages, e.g., C/C++, frames (a.k.a. **activation records**), are stored on a stack provided to the app by the operating system.
 - This stack is called the **system stack**.
- Whenever a C/C++ function is called, its activation record gets **pushed** onto the stack.
 - When a function returns, its activation record is **popped**.
- The activation record for the currently executing function is always on the top of the stack.

Depth of Recursion in Python

- Python monitors the number of recursive calls for any function.
- If recursion gets **too deep**, Python stops everything with a runtime error.
- Python's default is to limit recursion to about **1000 recursive calls**.
 - This reflects a conservative attitude about recursion.
 - Very often, if recursion reaches the limit, it's because of an error.
 - But not always!
 - This limit **can** be changed.

Depth of Recursion in Other Languages

- The system stack, is **limited in size**.
- If recursion gets **too deep** and uses up stack space, no more functions can be called.
- This is called a **stack overflow**, which results in a run-time error.
- Usually, **stack overflow results in catastrophic failure** of an app.
- Most languages do not limit recursion depth.
- It's your job to prevent stack overflow.

Python is not the whole story!

- Recursion in Python (and many languages) requires stack space.
- But the statement "Recursion always uses stack space" is **false**.
- Some languages can implement space efficient (i.e., $O(1)$) recursion. Python is not one of these.
- The technique is called **Tail-call optimization**
- It's good to be aware of the limitations of Python.
- It's also important not to over-generalize.

Time Complexity of Recursive Algorithms

- With loops:
 - Analyze the cost for body of the loop
 - Factor in the number of repetitions of the loop.
- With recursion:
 - Analyze the cost for body of the function, ignoring recursive calls
 - Factor in the number of times the function is called.

Example: Factorial

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

- Ignoring the function call, what's the cost of executing the function 1 time?
- How many times does the function get called?

Example: Sum of Squares

```
1 def sumsq(n):  
2     if n == 1:  
3         return 1  
4     else:  
5         return n*n + sumsq(n-1)
```

- Ignoring the function call, what's the cost of executing the function 1 time?
- How many times does the function get called?

Example: Exponentiation

```
1 def exp(r, n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return r * exp(r, n-1)
```

- Ignoring the function call, what's the cost of executing the function 1 time?
- How many times does the function get called?

Example: Exponentiation

We can do better, if we've studied a little math.

$$r^{2n} = r^n \times r^n$$

- Rewrite the recursive function to use this property!
- Figure out the time complexity of the new version.

Example: Binary search

```
1 def bin_search(C, target, start, end):
2
3     if end < start:
4         return False
5
6     mid = (start + end) // 2
7
8     if C[mid] == target:
9         return True
10    elif C[mid] < target:
11        return bin_search(C, target, mid+1, end)
12    else:
13        return bin_search(C, target, start, mid-1)
```

- Ignoring the function call, what's the cost of executing the function 1 time?
- How many times does the function get called?

Recursion

- It's fair to see recursion as just **another way to do repetition**
- We've seen no examples where **recursion is the only way**.
- But we will, right away!